Qwen

A Comprehensive Research Report on Irregular 3D Bin Packing for Organized Object Placement

This report provides a comprehensive analysis of the problem of packing irregular, undefined 3D objects into a rectangular bin with an emphasis on an organized placement strategy. The user's project involves using Python and common 3D file formats like STL and STEP to optimize for maximum object count within a fixed-size box. This report outlines the necessary software tools, computational strategies, algorithmic approaches, and advanced techniques required to build such a system. It deconstructs the user's "organized approach" into its fundamental components—bounding boxes, orientation management, level-based layering, and 2D packing sub-problems—and explores how these can be integrated into a sophisticated, high-performance solution. The analysis is based exclusively on the provided context blocks, ensuring fidelity to the source material while synthesizing disparate information into a cohesive guide.

Foundational Tools and Libraries for 3D Model Processing in Python

The initial and most critical step in any 3D bin-packing application is the ability to accurately read, parse, and analyze the geometric properties of the input 3D models. For a Python-based project involving irregular shapes from common CAD or mesh formats like STL and STEP, a robust ecosystem of libraries is available. These tools form the bedrock upon which all subsequent packing logic will be built, providing essential functionalities such as file loading, geometry manipulation, and core geometric calculations. The choice of library directly impacts the precision, performance, and capabilities of the final packing algorithm.

Two primary Python libraries are central to handling STL files, which are commonly used for representing triangular meshes of 3D objects: `numpy-stl` and `Trimesh`. The `numpy-stl` library (version 3.2.0) is specifically designed for high-performance reading, writing, and modification of binary and ASCII STL files [1]. Its reliance on `numpy` ensures that operations on large vertex sets are executed efficiently, making it ideal for computationally intensive tasks like packing [1]. This library supports fundamental geometric operations crucial for packing, including mesh rotation and translation, combining multiple STL files into a single mesh, and computing mass properties such as volume, center of gravity, and inertia tensor [1]. Furthermore, it can easily calculate the axis-aligned bounding box (AABB) of an object by simply finding the minimum and maximum coordinates across all vertices along the X, Y, and Z axes, a straightforward but vital calculation for any packing heuristic [1] [13].

While `numpy-stl` excels at low-level mesh data manipulation, `Trimesh` offers a more comprehensive and higher-level API for working with triangular meshes [2]. `Trimesh` is a pure Python library (Python 3.7+) that builds upon this foundation, providing a rich suite of analytical tools for watertight surface meshes [2]. It supports a wider array of file formats out of the box,

including STL, PLY, GLTF/GLB, and 3MF, and through external SDKs like GMSH, it can also import native CAD formats such as STEP, IGES, and BREP [2]. This makes it exceptionally versatile for projects dealing with both manufactured parts (STEP/IGES) and scanned models (STL). Like `numpy-stl`, `Trimesh` can compute AABBs, but it also introduces the capability to compute the minimum volume oriented bounding box (OBB), which can provide a tighter fit than the simpler AABB and potentially lead to more efficient packing arrangements [2]. Other key functions include slicing a mesh with a plane, computing the convex hull, and performing various mass property calculations, all of which are invaluable for complex packing scenarios [2]. Notably, `Trimesh` can also align the orientation of an object to a canonical state based on its principal components of inertia, a feature that directly supports the user's "organized" packing approach by establishing a consistent reference frame for each object [2].

For projects requiring parametric CAD modeling capabilities or needing to work with native CAD data structures beyond simple mesh conversion, CadQuery is an indispensable tool. Based on the powerful Open CASCADE Technology (OCCT) kernel, CadQuery is a Python framework for building precise, history-free 3D CAD models programmatically [6]. It allows for complex operations like extrusions, fillets, and nested assemblies to be scripted in Python. Crucially, CadQuery can import and export standard 3D file formats like STEP and STL, bridging the gap between parametric design and mesh-based computation [6]. This means a user could start with a parametric model, perform transformations, and then export it to a mesh format for use with `Trimesh` or `numpy-stl` for the actual packing simulation. With a strong community, active development, and a permissive Apache Public License, version 2.0, CadQuery provides a mature and scalable backend for advanced CAD-driven applications [6].

Finally, another potential tool worth mentioning, especially for medical imaging or scientific visualization contexts, is 3D Slicer. While not primarily a CAD or packing tool, its scripting environment allows for powerful geometric analysis. A Python script within 3D Slicer can load a segmented volume, convert it to a labelmap, and then compute an AABB by analyzing the non-zero voxels in the corresponding NumPy array [7]. This demonstrates a voxelization-based method for bounding box calculation, an alternative approach to direct mesh analysis. However, for general-purpose industrial or logistics packing, `Trimesh` and `CadQuery` are likely better suited due to their native support for CAD and mesh formats and their focus on geometric algorithms rather than image processing [2] [6] [7].

Library / Tool	Primary Functionality	Supported File Formats	Key Features Relevant to Packing
numpy-stl	Reading, writing, and modifying binary/ ASCII STL files with numpy performance.	Binary/ASCII STL [1]	Mass property calculation, mesh rotation/translation, AABB computation [1] [13].
Trimesh	High-level analysis of triangular meshes; pure Python.	STL, PLY, GLTF/GLB, 3MF, STEP, IGES, BREP via GMSH [2]	OBB computation, convex hull, principal component alignment, extensive analysis tools [2].

Library / Tool	Primary Functionality	Supported File Formats	Key Features Relevant to Packing
CadQuery	Parametric CAD scripting based on OCCT.	STEP, STL [6]	Programmatic CAD model creation, assembly, and transformation; strong OCCT integration [6].
3D Slicer	Medical image visualization and analysis.	Volumetric data formats; can process STL/ SEGMENTATION nodes [7]	Voxel-based AABB computation from segmentation data [7].

Algorithmic Frameworks for Optimizing Packing Density and Time

Once the 3D models have been processed and their geometric properties are known, the next challenge is to devise an algorithmic framework to determine the optimal arrangement of these objects within the container. The user's goal of maximizing object count requires a strategy that fills space as efficiently as possible, while minimizing packaging time necessitates an algorithm that avoids excessive computational overhead. The provided sources describe several established algorithmic paradigms that can be adapted to this problem, ranging from classic heuristics to modern machine learning approaches.

A foundational technique for tackling complex combinatorial problems like bin packing is integer linear programming (ILP). An ILP formulation for 2D bin packing has been proposed where the master problem aims to minimize the total number of packings used, subject to constraints that ensure all items are covered [3]. The generation of new, valid packing patterns is handled by a subproblem, often solved using column generation, which maximizes dual costs under constraints that prevent item overlap and duplicates [3]. This decomposition approach is powerful and can theoretically find an optimal solution. However, its practical application is severely limited by scalability; the number of potential placement options grows exponentially with the size of the container and the number of items, making the method computationally infeasible for problems of the scale typical in logistics or manufacturing [3]. Therefore, while understanding the ILP model is valuable for grasping the theoretical limits of optimality, it is not a viable approach for the user's project without significant simplification or approximation.

Moving to more practical and widely used heuristics, the concept of a layer-building algorithm is particularly relevant to the user's "organized" approach. This strategy involves creating horizontal layers of items and then stacking these layers within the bin [9] [12]. In one implementation, a constructive heuristic builds layers of identical items first, and then generates packing patterns by greedily loading these pre-made layers [12]. Another matheuristic approach uses Mixed Integer Linear Programming (MILP) to generate high-quality pallet configurations by sequentially solving two MILP sub-problems: one to minimize unused space within each layer, and another to minimize the number of layers needed overall [16]. This layered approach is highly effective because it reduces the

complexity of the 3D problem. By optimizing the 2D layout of a single layer first, the 3D problem becomes a series of 2D bin packing problems stacked vertically. This is precisely the strategy implied by the user's request to treat the problem as "2D packing but overthinked." Such methods have demonstrated remarkable success, achieving average bin fill ratios of up to 99% on real-world industrial datasets when combined with optimization solvers [16].

Beyond static heuristics, reinforcement learning (RL) represents a cutting-edge frontier for solving dynamic packing problems. Software like DeepPack, developed by InstaDeep, uses RL to learn an optimal policy for determining the sequence in which to pack items and for placing them in 3D space [4]. Similarly, DeepPack3D implements a similar RL approach alongside other classical heuristics like Best Area Fit (BAF), Best Shortest Side Fit (BSSF), and Best Longest Side Fit (BLSF) [10]. These RL models are trained in a simulated environment to maximize a reward function, typically defined as the amount of space filled or the stability of the resulting stack. The advantage of this approach is its ability to discover novel and highly efficient packing sequences that might not be intuitive to human designers or traditional algorithms. However, implementing RL requires significant expertise in machine learning, access to substantial training data or simulation environments, and specialized dependencies like TensorFlow [10]. While this is an open area for research, it may be too advanced for a project focused on immediate, practical implementation [4].

For a project seeking a balance between performance, ease of implementation, and effectiveness, a hybrid approach appears most promising. One could combine a greedy layer-building algorithm, inspired by the works of Saraiva et al. [12] and Dotoli et al. [16], with a fast and efficient 2D bin packing algorithm for arranging items within each layer. This would provide a strong baseline solution. To further enhance this, one could integrate a simple RL model or a genetic algorithm to optimize the order in which items are selected for packing, as suggested by the outer heuristic framework in the 3D-BDPP study [11]. This layered, multi-stage strategy respects the user's desire for an "organized" system while leveraging proven, scalable algorithms to achieve high packing densities.

Implementing an Organized Packing Strategy: Layering and Level-Based Stacking

The user's specification for an "organized" packing approach, characterized by objects placed in the same orientation with specific margins and separated by levels, provides a clear and implementable structure for the algorithm. This approach transforms the complex 3D problem into a more manageable hierarchical process, primarily by enforcing constraints on orientation and spatial arrangement. By breaking down this strategy, we can identify the core components and the algorithms needed to execute it effectively.

The first pillar of this organized strategy is the constraint on orientation. The user specifies that objects should be placed in the "same orientation." This significantly simplifies the problem by removing the need for a computationally expensive search over all possible rotations. In practice, this means that each object will have a predefined coordinate system, and during packing, it will only be translated, not rotated. This is a common assumption in many industrial applications where rotational freedom is physically constrained. The `Trimesh` library is particularly well-suited here, as it can align the orientation of an object to a canonical state based on its principal axes of inertia [2]. This establishes a consistent and predictable reference frame for every object, which is essential for

deterministic placement rules. Alternatively, one could define a fixed orientation based on the object's axis-aligned bounding box (AABB) dimensions, perhaps choosing the orientation with the smallest base area to create stable, flat-bottomed layers [13].

The second pillar is the use of levels or layers. This is a direct application of the layer-building heuristics described in the literature [12] [16]. The 3D bin is conceptualized as a stack of 2D planes, or levels. Items are packed into a single level until no more can fit, at which point a new level is started above it. This approach is inherently "organized" as it imposes a regular, structured pattern on the random-like placement problem. Each level forms a coherent unit, which can simplify downstream processes like robotic picking or inventory management. The height of each level is determined by the Z-dimension of the packed items within it. A key consideration in this strategy is the spacing between items. The user mentions a "specific margin around 3 sides of the bounding box," which translates to a small clearance distance between adjacent objects. This clearance prevents items from touching, which can be important for physical handling, thermal management, or avoiding damage. When implementing the 2D packing algorithm for each level, this margin must be incorporated by reducing the available space in the current level by the width of the margin times the number of gaps created by the placed items.

To operationalize this, the algorithm can proceed in a loop: 1. Initialize: Start with an empty bin and set the current vertical position (Z-coordinate) of the first level to zero. 2. Level Packing Loop: * Create a 2D representation of the current level, bounded by the bin's X and Y dimensions and located at the current Z-position. * Select a subset of the remaining unpacked items to place on this level. * Apply a 2D bin packing algorithm (e.g., Next-Fit Decreasing Height, MaxRects, or a best-fit heuristic) to arrange these items within the 2D boundary, respecting the required margins. * If no items can be placed on the current level, terminate the loop. * For each item placed, translate its 3D model to its computed (X, Y, Z) position in 3D space. * Remove the placed items from the list of unpacked items. * Update the current vertical position for the next level by adding the maximum Z-dimension of the items just placed plus the required inter-level spacing. 3. Termination: The algorithm ends when all items have been successfully packed into levels or no more items can be accommodated.

This level-based approach has several advantages. It naturally handles the user's requirement for separation by levels. It also allows for the imposition of additional constraints at the layer level, such as load stability or weight distribution, as explored in the work of Dotoli et al., who considered parameters like max area gap between consecutive layers and max weight per layer [16]. For example, one could enforce that the total weight of a level does not exceed a certain threshold (e.g., U = 10,000 mm² in the cited work) to prevent pallet collapse [16]. By focusing the complex 3D optimization on a series of simpler 2D problems, this organized strategy provides a robust and scalable path toward achieving high packing densities in a predictable and controllable manner.

Solving the 2D Sub-Problems: Core Algorithms and Techniques

At the heart of the organized, level-based packing strategy lies the 2D bin packing problem: given a rectangular bin and a set of 2D items (the top-down projections of the 3D objects), how can the items be arranged to minimize wasted space? The efficiency of the overall 3D packing algorithm is heavily dependent on the quality of the solution to this 2D sub-problem. The provided sources

describe several classes of algorithms for this task, ranging from simple greedy heuristics to more complex shelf-based methods.

One class of algorithms modifies the basic "level algorithms" popularized by Coffman, Garey, Johnson, and Tarjan [15]. These algorithms work by maintaining a set of horizontal "shelves" or levels within the bin. New items are placed onto an existing shelf if they fit, or a new shelf is created above the highest existing one. The paper by Baker and Schwarz introduces modifications to these classical shelf algorithms, proposing variants called "next-fit" and "first-fit" shelves [15]. Unlike some level algorithms that require items to be sorted beforehand by height, these modified versions do not, which can reduce preprocessing time. They introduce a parameter r that controls the shelf height, and by choosing r appropriately, the worst-case performance of these algorithms can be made to approach that of the classical level algorithms [15]. Analyzing the non-asymptotic worst-case performance provides a guarantee on the solution quality, even if it is not always optimal [15]. These algorithms are relatively simple to implement and serve as a strong baseline for the 2D layer packing.

A more sophisticated approach to the 2D sub-problem involves the use of greedy algorithms, which make the locally optimal choice at each step. For instance, one could implement a "best-fit" heuristic for each item. For each item to be placed, the algorithm would iterate through all currently occupied shelves to find the lowest shelf where the item fits horizontally without causing an overhang that violates constraints. If no such shelf exists, a new shelf is created. Within this process, different selection criteria can be applied to choose which item to place next. The layer-building algorithm by Saraiva et al. uses different selection criteria to generate packing patterns, demonstrating that the choice of what to pack next is as important as where to pack it [12]. For example, one could prioritize items by area, perimeter, or aspect ratio to improve packing density. This flexibility allows for fine-tuning the algorithm to better suit the specific characteristics of the object set.

Another powerful technique for solving the 2D sub-problem is the use of constraint programming or exact methods, though these are typically reserved for smaller instances due to their computational cost. The integer linear programming (ILP) model for 2D bin packing, for example, can be applied to the 2D layer packing problem [3]. In this case, the "bin" is the current level, and the "items" are the 2D projections of the unpacked objects. The master problem would still aim to cover all required items, while the subproblem would generate new valid 2D packing patterns. While this guarantees an optimal solution for the 2D layer, the exponential growth in the number of possible placements remains a concern [3]. For a project aiming to balance accuracy and speed, a hybrid approach seems most appropriate: use a fast greedy heuristic like Best Fit for the bulk of the packing, and for the last few items, one could switch to a more exhaustive search or even solve the final 2D problem using an ILP solver to squeeze out the last bit of efficiency.

In practice, a combination of these techniques is often employed. A common method is the Maximal Rectangles algorithm, which maintains a list of maximal free rectangles (rectangles that cannot be expanded further without overlapping an occupied space). When placing a new item, it is positioned in one of these free rectangles in a way that optimizes a certain criterion, such as touching the bottom and left edge (Bottom-Left rule). After placing the item, the list of free rectangles is updated by carving out the new occupied space and merging adjacent free spaces back into larger ones. This algorithm is efficient, easy to understand, and works well with constraints like margins. Given the user's preference for a structured, "organized" solution, a shelf-based algorithm enhanced with a

good item selection heuristic is likely the most suitable choice for the 2D sub-problems, providing a reliable and reasonably efficient method for populating each level.

Advanced Strategies for Maximizing Space Utilization and Efficiency

While the organized, level-based approach provides a solid foundation, pushing towards maximum space utilization and computational efficiency requires the adoption of more advanced strategies. These strategies go beyond simple heuristics and involve sophisticated optimization techniques, machine learning, and the careful management of real-world constraints. By integrating these methods, it is possible to move from a good solution to a near-optimal one, tailored to specific logistical requirements.

One of the most effective advanced strategies is the use of hybrid algorithms that combine the strengths of different methods. For example, a matheuristic approach, as described in the work on the 3D-Single Bin-Size Bin Packing Problem (3D-SBSBPP), pairs mathematical programming with heuristic techniques [16]. Their framework uses two sequential Mixed Integer Linear Programming (MILP) sub-problems to first optimize the configuration of a single layer (minimizing waste and gaps) and then to determine the overall bin configuration (minimizing the number of layers) [16]. This structured, multi-stage optimization proved highly effective, achieving very high bin fill ratios (up to 99%) on industrial data [16]. A similar philosophy can be applied to the user's project: after generating a set of candidate 2D layer layouts using a fast greedy algorithm, a lightweight optimization step could be introduced to fine-tune the positions of a few items on the layer to close small gaps. This keeps the majority of the computation fast while allowing for minor improvements.

The second major avenue for advancement is the application of artificial intelligence and machine learning, particularly reinforcement learning (RL). Traditional heuristics rely on pre-defined rules for item selection and placement. RL, in contrast, learns an optimal policy from experience. Software like DeepPack and DeepPack3D trains an agent to solve the 3D bin packing problem by rewarding it for successful packings and penalizing it for wasted space or unstable stacks [4] [10]. The RL agent learns to predict the best next action (which item to place and where) based on the current state of the bin. This can lead to discovering highly efficient, non-intuitive packing sequences that outperform hand-crafted heuristics [4]. DeepPack3D, for instance, was developed for robotic palletization systems and integrates GPU acceleration to speed up the RL inference process [10]. While implementing RL is more complex than coding a heuristic, it represents a state-of-the-art approach for achieving superior results in dynamic and complex packing scenarios.

A third advanced strategy is the explicit incorporation of real-world constraints into the packing model. The user's "organized" approach already touches on this by enforcing uniform orientation and spacing. However, the literature describes a much richer set of constraints that can be modeled. The matheuristic from Dotoli et al. explicitly considers load bearing strength, stability, height homogeneity, and weight limits for each layer [16]. For example, they impose a maximum overhang (O and Q) to ensure stability and a maximum height gap (G) between layers to allow for easy robotic picking [16]. These constraints are represented by parameters (FRmono, FRmulti, G, B, etc.) in their MILP formulation. By adopting a similar mindset, the packing algorithm can be tailored to meet specific logistical needs, whether it's ensuring that heavy items are at the bottom or that all items in a

layer are accessible from the same side. This transforms the abstract problem of maximizing volume into a concrete, application-specific optimization task.

Finally, optimizing the packing sequence itself is a critical factor. The order in which items are presented to the packing algorithm can dramatically affect the final result. A simple greedy algorithm might fail to place a large item late in the process if smaller items have already blocked the best locations. To address this, metaheuristic algorithms like genetic algorithms or differential evolution can be used to evolve a sequence of items that leads to a high-quality packing [11]. The inner constructive heuristic (the actual packing algorithm) acts as a fitness function, evaluating how well a given sequence packs the items. This outer optimization loop, as seen in the 3D-BDPP study, can iteratively improve the packing result by exploring different item orders [11]. Combining this sequencing optimization with a layered packing strategy creates a powerful, multi-faceted solution that addresses the problem from multiple angles simultaneously.

Performance Optimization and Practical Implementation Considerations

Developing a 3D bin packing algorithm that is both accurate and fast enough for practical use requires careful attention to performance optimization and implementation details. The choice of programming language, data structures, and algorithmic shortcuts can mean the difference between a tool that provides instant feedback and one that is too slow for real-world deployment. The user's goal of minimizing packaging time is a critical non-functional requirement that must be addressed throughout the development lifecycle.

First, the selection of the right Python libraries and tools is paramount for performance. As established, `Trimesh` and `numpy-stl` are excellent choices for geometric processing because they are built on top of optimized C/Fortran backends (like NumPy) and are designed for high-performance numerical computation [1] [2]. Using these libraries for core tasks like loading STL files, computing bounding boxes, and performing transformations ensures that the most common operations are executed efficiently. For the packing logic itself, a well-optimized implementation of a 2D bin packing heuristic, such as the Maximal Rectangles algorithm, can be written in pure Python but should leverage NumPy for managing the state of the bin (e.g., representing the bin's surface as a 2D grid or using NumPy arrays to track free space). Profiling the code with tools like Python's built-in `cProfile` is essential to identify and eliminate bottlenecks.

Second, algorithmic pruning and early termination are crucial for managing runtime, especially for more complex algorithms. Even if a full ILP or RL solution is too slow, a branch-and-bound style approach can be implemented to find a high-quality solution quickly and then prune branches of the search tree that cannot possibly lead to a better result. For greedy heuristics, this can be as simple as setting a maximum runtime and returning the best solution found so far. The Matheuristic approach by Dotoli et al. provides a great example of this trade-off, with a parameter (Δt) explicitly limiting the solver time per layer to 90 seconds to ensure computational efficiency [16]. This pragmatic approach allows the algorithm to produce a very good solution without getting bogged down in a lengthy search for a marginal improvement.

Third, parallelization and hardware acceleration offer significant potential for speeding up the computation. Some aspects of the packing problem are inherently parallelizable. For instance, when evaluating all six possible orientations for a rectangular component to find the one that maximizes packing density, each orientation can be evaluated independently [13]. Similarly, if the outer optimization loop is testing multiple item sequences in parallel, each sequence can be processed concurrently. For machine learning approaches like DeepPack3D, GPU acceleration is not just beneficial but essential for practical performance, as it massively speeds up the matrix multiplications involved in neural network inference [10]. While a project might start without GPUs, planning for future scalability by using frameworks like TensorFlow or PyTorch can pay dividends later [10].

Fourth, the implementation architecture plays a role in performance and maintainability. A modular design separates concerns, making the system easier to test, debug, and optimize. For the user's project, a logical architecture could consist of: * Data Layer: Responsible for loading and converting 3D models using libraries like `Trimesh` and `CadQuery`. * Analysis Layer: Computes and stores geometric properties like AABBs and principal axes [2] [13]. * Algorithm Layer: Contains the core packing logic, including the 2D sub-algorithms and the layer-building strategy. * Optimization Layer: Implements advanced strategies like sequence optimization or RL agents [4] [11]. * Interface Layer: Manages user interaction, either through a command-line interface or a GUI built with a framework like Tkinter [13].

Finally, benchmarking and validation against real-world data are essential. Theoretical performance is not enough. The algorithm's efficiency and effectiveness must be tested on realistic datasets, ideally sourced from the specific application domain. The study by Dotoli et al. validates its matheuristic on both classical benchmarks and a case study from an e-commerce company, deriving valuable managerial insights in the process [11]. Similarly, testing the Productive Packager on benchmark instances showed packing efficiency improvements of 8.55% to 30% over manual methods [13]. By comparing the output of the developed system against known solutions or industry standards, developers can quantify its performance and identify areas for improvement.

Reference

1. numpy-stl - PyPI https://pypi.org/project/numpy-stl/
2. trimesh 4.7.4 documentation https://trimesh.org/
3. 2D bin packing with predefined gaps in container - Stack Overflow https://stackoverflow.com/questions/46630524/2d-bin-packing-with-predefined-gaps-in-container
4. Python example of 3D bin packing problem and visualization https://stackoverflow.com/questions/68953770/python-example-of-3d-bin-packing-problem-and-visualization
5. enzoruiz/3dbinpacking: A python library for 3D Bin Packing - GitHub https://github.com/enzoruiz/3dbinpacking
6. CadQuery/cadquery: A python parametric CAD scripting ... - GitHub https://github.com/CadQuery/cadquery

7. Python script to get axis-aligned bounding box - 3D Slicer Community https://discourse.slicer.org/t/python-script-to-get-axis-aligned-bounding-box/26227
8. Axis-Aligned Bounding Box Calculation (AABB) for different ... https://stackoverflow.com/questions/74358413/axis-aligned-bounding-box-calculation-aabb-for-different-orientations-of-3d-ob
9. A three-stage layer-based heuristic to solve the 3D bin-packing ... https://www.sciencedirect.com/science/article/pii/S1319157821001749
10. DeepPack3D: A Python Package for Online 3D Bin Packing ... https://codeocean.com/capsule/2079012/tree
11. (PDF) Two-layer Heuristic for the Three-Dimensional Bin Design ... https://www.researchgate.net/publication/374556011_Two-layer_Heuristic_for_the_Three-Dimensional_Bin_Design_and_Packing_Problem
12. A layer-building algorithm for the three-dimensional multiple bin ... https://www.sciencedirect.com/science/article/abs/pii/S2405896315003687
13. [PDF] Efficient Component Packing with Algorithmic Optimizations https://jisem-journal.com/index.php/journal/article/download/1240/471/2047
14. binpacking · PyPI https://pypi.org/project/binpacking/
15. Shelf Algorithms for Two-Dimensional Packing Problems - SIAM.org https://epubs.siam.org/doi/10.1137/0212033
16. Automating Bin Packing: A Layer Building Matheuristics for Cost ... https://ieeexplore.ieee.org/document/9787801/